Project 31-bis: Word Sense Disambiguation in Medical Context

Comparing Lesk Variants and a Phonetic–Semantic Hybrid

Project Leader: Giorgio Alleva [2605058]
Group Members: Francesca Lonardo [2605068]
Natural Language Processing & Text Mining
University of Oulu
Source Code and Data Instructions: https://github.com/pygiopoli/nlp_project_26

***Abstract*—Word Sense Disambiguation (WSD) aims to identify the intended meaning of an ambiguous word in context, a particularly challenging task in the biomedical domain, where both regular terms and acronyms map to multiple clinical concepts. In this project, we evaluate knowledge-based WSD methods on the MSH-WSD dataset, reporting performance separately for acronyms and regular terms. To evaluate standard WordNet-based implementations in NLTK and pywsd against the UMLS concepts used as ground truth in MSH-WSD, we first construct a mapping between UMLS concepts and WordNet synsets. Baseline experiments with the standard Lesk algorithm yield an accuracy of 0.25 on acronyms and 0.43 on regular terms: errors on acronyms stem primarily from missing medical senses in WordNet, whereas errors on regular terms reflect the limitations of Lesk. Stopword removal and sense-signature enrichment improve the baseline, although the pywsd algorithms remain below 0.40 accuracy on both subsets. We further explore whether phonetic and orthographic similarity, based on Soundex codes and edit distance, can help, integrating them both with the Wu–Palmer similarity and as an approximate word-matching criterion inside Lesk. Both strategies yield minimal gains (at most 0.017 accuracy): Soundex codes are too simple to separate senses, while edit distance only catches a few spelling variants. In contrast, a domain-specific approach based on SapBERT embeddings and UMLS definitions removes the dependency on WordNet and clearly outperforms the symbolic methods, reaching 0.88 accuracy on acronyms and 0.71 on regular terms. Finally, we developed a web interface to compare the methods on any sentence.**

***Index Terms*—Word Sense Disambiguation, Lesk, MSH-WSD, biomedical NLP, WordNet, Wu–Palmer, phonetic similarity, Soundex, SapBERT, UMLS.**

I. Introduction

Word Sense Disambiguation (WSD) is the task of identifying which meaning of an ambiguous word is used in a given context, and it is a long-standing problem in Natural Language Processing [1]. In the biomedical domain, it is particularly important, since the same term can refer to different concepts (the word *cold* can be interpreted as a temperature or as a disease), and acronyms are extremely ambiguous (*CAD* can mean *coronary artery disease* or *computer-aided diagnosis*). Since WSD is a key step of larger systems, its errors propagate to information extraction, indexing, and retrieval of medical literature [2]. In this domain, the meanings are usually defined by large knowledge sources such as the Unified Medical Language System (UMLS) [3], which integrates the concepts of many medical vocabularies and identifies each of them with a unique code (CUI).

Among knowledge-based methods, the Lesk algorithm [4] chooses the sense whose dictionary definition has the most words in common with the context. Since the original version compares the definitions of all the words in the context, most implementations use the simplified Lesk [5], which compares each definition directly with the context, and many extensions enrich the signature of a sense with related WordNet senses [6]. Another family of methods measures the semantic similarity between senses in the WordNet hierarchy, for example with the Wu–Palmer similarity [7]. Libraries such as NLTK [8] and `pywsd` [9] make these algorithms easy to use, but they all rely on WordNet [10], a general-purpose lexicon with limited medical coverage.

The MSH-WSD dataset [11] was built automatically from MEDLINE abstracts indexed with MeSH headings, and contains 203 ambiguous terms and abbreviations, each labelled with UMLS concepts. On this dataset, knowledge-based methods that use UMLS instead of WordNet obtain good results: a Lesk-like approach based on UMLS definitions reaches about 0.81 accuracy, while supervised classifiers reach 0.94 [11]. Comparisons of knowledge-based approaches show that they remain below supervised methods and that combining them helps [12]. More recent work represents contexts and concepts with distributed representations: comparing the embedding of the context with embeddings of UMLS definitions gives about 0.84 accuracy on MSH-WSD [13], and domain-specific models such as SapBERT [14], a BERT model [15] trained to align UMLS synonyms, provide strong concept representations.

In this project, we study how classical WordNet-based algorithms behave on medical text and whether they can be improved. This raises several challenges: (i) the correct meanings in MSH-WSD are UMLS concepts, while Lesk-based methods output WordNet senses, so the two inventories must be linked; (ii) acronyms and regular terms behave very differently and must be evaluated separately; (iii) the many algorithms of `pywsd` must be compared on the same data; (iv) we propose to integrate *phonetic similarity*, based on Soundex codes [16] and edit distance [17], into similarity-based Lesk to handle spelling and morphological variations; finally, (v) the results must be presented to a user through a

graphical interface.

The main novelty of our work is the combination of phonetic and semantic similarity for WSD: phonetic matching is commonly used for name matching and spelling correction, but it is rarely applied to sense disambiguation. We test it with the Wu–Palmer similarity and as an approximate word matching inside Lesk. As a further extension, we compare these methods with a neural approach based on SapBERT, which does not depend on WordNet.

II. Methodology

A. Problem Definition

Given an abstract containing an ambiguous biomedical target word, the goal is to identify which UMLS concept (CUI) corresponds to the intended meaning. A central challenge of our setup is the mismatch between sense inventories: while the ground-truth annotations in MSH-WSD are UMLS concepts, the knowledge-based algorithms in NLTK and `pywsd` operate on WordNet synsets. Therefore, before evaluating any disambiguation method, the two inventories must be aligned (Section II-C). All the code is in the `scripts` folder of our repository.

B. Dataset

The MSH-WSD dataset [11], created at the U.S. National Library of Medicine and obtained from the `yarn` repository [13], comprises 203 ambiguous biomedical words. The file `benchmark_mesh.txt` lists the candidate CUIs for each word, while individual word files contain the corresponding MEDLINE abstracts, in which the target word is marked and annotated with the index of its CUI. Because the dataset does not explicitly label which entries are acronyms, we classify a target word as an acronym if it contains at least two uppercase letters or a digit, manually correcting a few edge cases (e.g. *Ca* and *Fe* are chemical abbreviations, whereas *ERUPTION* is a regular term). Three words that function as both a regular term and an abbreviation (*EGG*, *STEM* and *Orf*) are excluded, yielding 108 acronyms and 92 regular terms. The original paper reports 106 abbreviations, 88 terms and 9 mixed cases: the total of 203 words is identical, and the difference comes only from how the mixed cases are assigned.

We use the subsets of Table I. For Task 2, as required, we randomly select 10 acronyms and 10 terms (seed 42). Since many of them have zero or one WordNet sense, we draw a second selection (seed 7) only among the words with at least two WordNet senses. This is a necessary condition for a real choice, not a sufficient one, since the senses of a word may not be medical. The union of the two selections (39 words, since *Cilia* appears in both) is used for Tasks 4, 6, 10 and 11, with 10 abstracts per sense (seed 0, 810 abstracts in total). Finally, the methods of Tasks 4 and 6 (Task 9) and of Task 11 are also run on all 200 words, with the same sampling of 10 abstracts per sense.

TABLE I
Subsets of MSH-WSD Used in the Experiments

Subset	Acronyms	Terms	Tasks
Selection 1 (random)	10	10	2
Selection 2 ($\geq$2 senses)	10	10	2
Union of the selections	20	19	4, 6, 10, 11
Full dataset	108	92	9, 11

C. CUI–WordNet Mapping

For the 39 sampled words (the union of the two selections, Section II-B), we constructed the sense mapping manually. For each word, we retrieve the preferred names of its candidate CUIs via the UMLS REST API alongside its WordNet synsets, associating each CUI with the semantically equivalent WordNet synset(s). We enforce two strict constraints: (i) a CUI lacking an equivalent WordNet sense is assigned an empty list, meaning that WordNet-based algorithms cannot correctly disambiguate its occurrences; and (ii) each WordNet synset is mapped to at most one CUI, so that a single synset cannot be counted as correct for every sense of the word, which would give perfect accuracy without any disambiguation.

Because manual annotation is infeasible for the full 200-word dataset, we designed an automatic mapping procedure. For each CUI, we fetch its synonymous names and English definitions from UMLS and link it to a candidate WordNet synset if either (Rule 1) one of the UMLS names exactly matches a lemma of the synset, or (Rule 2) its names and definitions share at least a minimum number of content words with the gloss, lemmas and hypernyms of the synset. This minimum was tuned against the 39 manually mapped words (Section III-D).

D. Evaluation Protocol

For each target word, we define three evaluation metrics: (i) *accuracy*, the proportion of correctly disambiguated abstracts; (ii) *coverage*, the proportion of abstracts whose CUI maps to at least one WordNet synset, which is the upper bound for any WordNet-based method; and (iii) *accuracy on covered senses*, the accuracy restricted to the covered abstracts. While accuracy evaluates the whole system, accuracy on covered senses isolates the disambiguation capability of the algorithm from the coverage of WordNet. Metrics are macro-averaged across words, giving equal weight to each target word, and reported separately for acronyms and regular terms. As a reference, we also compute the expected accuracy of a random choice among the candidate WordNet senses.

E. Task 1: Lesk on an Example Sentence

We first evaluate the standard NLTK Lesk on the sentence "I have been prescribed two important drugs today during my visit to clinic" (target *drug*), in four configurations: without restrictions, restricted to noun senses, restricted to the part of speech predicted by the NLTK tagger, and with our extended Lesk. Following Banerjee and Pedersen [6], our extended Lesk lemmatizes both context and glosses, removes stopwords, and

expands each sense signature with the lemmas, definitions and usage examples of related synsets (hypernyms, hyponyms up to depth two, meronyms, holonyms and similar senses).

F. Task 2: Lesk on MSH-WSD

We first apply the standard Lesk algorithm of NLTK to the random selection of 10 acronyms and 10 terms. For each abstract, the context is the whole abstract (lowercased and split into words), the candidates are the WordNet senses of the target word, and the answer is correct if the chosen synset is one of the synsets mapped to the correct CUI.

Many words of this first selection have zero or one WordNet sense, so Lesk either cannot answer or has nothing to choose. To evaluate the algorithm in cases where it really has to choose, we repeat the experiment on the second selection, drawn only among the words with at least two WordNet senses (Section II-B).

We then try to improve Lesk with two simple changes, tested alone and together. The first is *stopword removal*: we remove from the context the English stopwords of the NLTK list (e.g. *the*, *of*, *to*). Since stopwords appear in every abstract and in many glosses, they can give points to a sense without any relation to its meaning. The second is a *part-of-speech filter* on the candidates: instead of tagging the context, we exploit the fact that the ambiguous words of MSH-WSD are nouns or abbreviations, and we give Lesk only the noun senses of the target (we filter the candidate synsets ourselves and pass them through the synsets argument of nltk.wsd.lesk), discarding verb senses such as *milk.v.01* ("take milk from female mammals"), which are unlikely to appear in medical abstracts. If a word has no noun senses, all its senses are kept.

Finally, we compare the best of these versions with two alternatives, both using stopword removal and the noun filter. The first is the Most Frequent Sense (MFS) baseline, which always chooses the first WordNet sense: it tells us whether Lesk is better than a simple rule. The second is our own implementation of the simplified Lesk. Like NLTK, it chooses the sense whose *signature* (the set of words describing the sense) shares the most words with the context, but it builds the signature more carefully: the gloss is lowercased, punctuation and stopwords are removed, and the target word is removed too, since it appears in every abstract and would give a free point to every sense that contains it. We test two versions of the signature: the first contains only the gloss, as in NLTK; the second also contains the WordNet examples and synonyms of the sense (e.g. *lash* and *cilium* for *eyelash*), so that a sense is described by more words.

G. Tasks 3-4: `pywsd` Algorithms

We tested four Lesk variants provided by `pywsd` (Original, Simple, Adapted and Cosine Lesk), six semantic similarity measures based on the WordNet hierarchy (Path, Wu–Palmer [7], Leacock–Chodorow [18], Resnik [19], Jiang–Conrath [20] and Lin [21]), and the max lemma count baseline. All the similarity-based methods use the same algorithm (`max_similarity`): each sense of the target is scored by summing, over the context words, its highest similarity with their senses, and only the similarity measure changes.

In Task 3, we evaluate these methods on three sentences, each chosen for a different purpose: the sentence of Task 1 with target *drug*, to compare `pywsd` with the NLTK Lesk; the *bank* example of the `pywsd` documentation, to check that the library works as expected; and a medical sentence with target *ventricle*, whose two WordNet senses are both medical (heart and brain). For each method we recompute the score of every candidate sense, so that we can inspect the full ranking rather than only the chosen sense.

In Task 4 (and 9), we scale the evaluation to MSH-WSD using a context window of ± 10 words around the target. Because `pywsd` relies on Open English WordNet 2024 [22], its output synsets are aligned back to NLTK's WordNet 3.0 via sense keys, to ensure compatibility with our CUI mapping. Since pywsd lemmatizes the target word and is case-sensitive (e.g. ALS becomes al), for each word we try a few spellings (original, lowercase, capitalized, uppercase) and keep the one whose pywsd senses include the largest number of synsets of our mapping. Furthermore, we introduce a corrected implementation of Resnik similarity, having identified a bug in `pywsd` (function `max_similarity`) where the ranking for Resnik is sorted in ascending order, so that the function returns the candidate sense with the *minimum* rather than maximum similarity score.

H. Task 5: Phonetic Similarity (+ Extension)

To capture phonetic resemblance, each token is encoded via a simplified version of the American Soundex algorithm, which retains the initial letter and maps subsequent consonants to digits representing articulatory equivalence classes, yielding a four-character code (e.g., *two* and *to* both map to `T000`). As we encountered some problems with the standard python library (Soundex), we re-implemented that in a specific utilities module (also contained in the /scripts directory). Although the project guidelines mention the Python `Fuzzy` package, the library is no longer maintained and fails to build on modern Python 3 environments due to legacy C extensions; consequently, we implemented Soundex directly within our utils.py module to guarantee reproducibility, while the Levenshtein distance is computed with `nltk.edit_distance`. Our Soundex is a simplified version: it does not code twice two consonants with the same code separated by a vowel (Tymczak gives T520 instead of T522), and it does not skip a second letter with the same code as the first one (Pfister gives P123 instead of P236). We define the phonetic distance between two words as the Levenshtein distance d between their four-character Soundex codes, and their phonetic similarity as $1 - d/4$. For the diagnostic sentence of Task 1, we compute pairwise distance matrices across all tokens both at the surface-word level and at the Soundex-code level. To enrich the analysis, we decided to translate the analyzed sentence in foreign languages (Italian, Finnish, Spanish, French, German, Russian), to better visualize the differences between latin, germanic and slavic languages.

I. Task 6: Phonetic–Semantic Hybrid

The Wu–Palmer method of `pywsd` scores each sense s of the target by summing, over the context words, the best similarity between s and the senses s' of that word. We keep this procedure and replace only the similarity measure with a combination of a semantic and a phonetic component:

$$\mathrm{sim}(s, s') = \alpha\,\mathrm{WuP}(s, s') + (1-\alpha)\,\mathrm{Phon}(s, s'), \qquad \alpha \in [0, 1]. \tag{1}$$

Unlike `pywsd`, we exclude the target word from the context, to avoid favouring candidate senses that happen to be similar to other senses of the same lemma.

The phonetic component must operate at the sense level rather than the word level: comparing the target word itself with the context words would give the same score to all its candidate senses, leaving the ranking unchanged. For this reason we define the phonetic similarity between two senses as the highest similarity between their lemmas,

$$\mathrm{Phon}(s, s') = \max_{l \in s,\, l' \in s'} \left(1 - \frac{d(l, l')}{4}\right), \tag{2}$$

where d is the Levenshtein distance between the Soundex codes of the two lemmas. Since Soundex codes have four characters, $d \in [0, 4]$, so both components lie in $[0, 1]$ and no further normalisation is needed. For a lemma of several words, the Soundex code usually depends only on its first word. The full procedure is given in Algorithm 1.

Besides the required $\alpha = 0.5$, we test $\alpha = 1$, which uses only Wu–Palmer and allows us to compare our implementation with `pywsd`, and $\alpha = 0.8$, which gives a smaller weight to the phonetic component.

Algorithm 1 Phonetic–Semantic Hybrid WSD

Require: target w, context words $c_1, \ldots, c_n$, weight α
Ensure: predicted sense $\hat{s}$

remove w from the context words
for each sense s of w **do**
  $\mathrm{score}(s) \leftarrow 0$
  for each context word c_i **do**
    $best \leftarrow 0$
    for each sense s' of c_i **do**
      $W \leftarrow \mathrm{WuP}(s, s')$
      $P \leftarrow \mathrm{Phon}(s, s')$ ▷ Eq. (2)
      $best \leftarrow \max\big(best,\ \alpha W + (1-\alpha)P\big)$
    end for
    $\mathrm{score}(s) \leftarrow \mathrm{score}(s) + best$
  end for
end for
return $\hat{s} = \arg\max_s \mathrm{score}(s)$

J. Task 9: Full-Dataset Analysis

The 39 selected words give useful but noisy estimates. To check whether our conclusions also hold on a larger scale, we run the methods of Tasks 4 and 6 on all the 200 words of MSH-WSD, using the automatic mapping of Section II-C.

K. Task 10: Phonetic Lesk

In Task 6, phonetic similarity is added to Wu–Palmer and compares the names of two senses. Here we use it where it seems more natural: inside Lesk, to compare words. Lesk counts a word as shared by the context and the signature only if it is written in exactly the same way, so it misses spelling and morphological variants of the same word, such as *oedema* and *edema*.

We use a version of our simplified Lesk of Task 2: the signature of a sense contains its WordNet definition, examples and synonyms, without stopwords, words shorter than three letters and the target word, and the context is the set of words of the whole abstract, cleaned in the same way. Unlike Task 2, all the WordNet senses of the target are candidates (no noun filter). The score of a sense is the number of context words that match at least one word of its signature. We change only the rule that decides when two words match:

- **Exact:** the two words are identical, as in Task 2;
- **Soundex:** the two words have the same Soundex code [16], i.e. they sound alike;
- **Levenshtein:** the two words differ by at most one edit (Levenshtein distance [17] ≤ 1). We apply this rule only to words of at least five letters, since in short words a single letter often changes the meaning (e.g. *milk* and *mild*).

We also remove the target word from the context, because otherwise Soundex would match it with unrelated words of the signatures in every abstract (e.g. *cilia* and *cell* share the code C400).

Before the evaluation, we tested the three rules on nine pairs of words: four variants of the same word (e.g. *haemorrhage*/*hemorrhage*, *eyelid*/*eyelids*), which should match, and five different words that look or sound similar (e.g. *drug*/*dirge*, *heart*/*hurt*), which should not. We then ran the three versions on the 39 selected words, with the same sample of abstracts as Tasks 4 and 6 (10 per sense, 810 abstracts) and the manual mapping, and computed accuracy and accuracy on covered senses. Finally, we counted the abstracts in which Levenshtein changes the answer of exact matching.

L. Task 11: SapBERT extension and neural approach

To avoid the limited coverage of WordNet, we also test a neural approach that chooses directly among the UMLS concepts. We use SapBERT [14], a biomedical BERT model further trained on UMLS so that the synonyms of the same concept have close vectors. For each CUI we build a short description with its five shortest UMLS names and its first English definition, and we encode it with the vector of the model's `[CLS]` token. Each occurrence of the target is encoded in the same way, using the ± 10 words around it (on the same sample as Tasks 4 and 6: 10 abstracts per sense, seed 0, 4,170 abstracts over the 200 words; inputs are truncated to 128 tokens), and we choose the CUI whose vector has the highest cosine similarity with the context. The method is *zero-shot*: the model is used as it is, without any training on the labelled abstracts of MSH-WSD, so it should be compared with the knowledge-based methods rather than with supervised ones. To see which choices matter, we also test larger windows (± 25

TABLE II
LESK VERSIONS ON THE SENTENCE OF TASK 1 (TARGET *drug*)

Version	Chosen sense	
Standard Lesk	drug.v.01 (administer a drug to)	
Standard Lesk, nouns only	drug.n.01 (medicine or narcotic)	✓
Standard Lesk + POS tagger	drug.n.01 (medicine or narcotic)	✓
Extended Lesk	drug.n.01 (medicine or narcotic)	✓

TABLE III
STANDARD LESK ON THE TWO SELECTIONS OF TASK 2

Selection	Group	Acc.	Cov.	Acc. cov.	Random
1 (random)	Acronyms	0.248	0.300	0.847	–
	Terms	0.434	0.574	0.825	–
2 ($\geq$2 senses)	Acronyms	0.192	0.283	0.607	0.383
	Terms	0.403	0.950	0.453	0.278

and ± 50 words) and descriptions made only of the names or only of the definition.

M. Task 7: Graphical User Interface

To make our disambiguation pipeline interactively accessible, we developed a web application using the Django framework. The Python backend invokes the exact disambiguation modules used in our work, while the front end is styled with Bootstrap5 (CDN based, we preferred to keep the repository as light as possible). Users can input a custom biomedical sentence, specify a target ambiguous word, select among the implemented algorithms (NLTK Lesk variants, `pywsd` methods, our Phonetic–Semantic hybrid, and SapBERT), and inspect both the predicted sense (with its definition) and the full candidate ranking scores. The application is located inside the `dis_project` directory of our repository. After activating the virtual environment (`.venv/Scripts/activate`) and can be launched locally in development mode via the standard command `python manage.py runserver`.

III. RESULTS AND DISCUSSIONS

A. Task 1: Lesk on an Example Sentence

WordNet has three senses of *drug*: one noun ("a substance that is used as a medicine or narcotic") and two verbs ("administer a drug to", "use recreational drugs"). Table II shows the answers of the four versions. The standard NLTK Lesk chooses the wrong sense *drug.v.01*. With the raw tokens the noun sense shares no word with the sentence, while "administer a drug **to**" shares the stopword *to* and "use recreational **drugs**" shares *drugs*: the two verbs tie with one point each and the first one is returned. The result therefore depends on a stopword and on the order of the senses, not on the meaning of the sentence.

Restricting Lesk to nouns gives the correct sense, but only because *drug* has a single noun sense, so no real choice is made; the POS tagger correctly tags *drugs* as a noun, so this restriction can be obtained automatically. Our extended Lesk also finds the correct sense, even without the POS filter, but with weak evidence: its score is 1 (the word *two*, found in the extended gloss) against 0 for the verbs. Overall, Lesk is very fast (one comparison per sense), but on a short sentence the overlaps are so few that the answer is close to a guess.

B. Task 2: Lesk on MSH-WSD

Table III reports the results of the standard NLTK Lesk on the two selections. On the random selection Lesk reaches 0.248 on acronyms and 0.434 on terms. The main cause of the difference is the coverage: WordNet contains the correct medical meaning in only 30% of the acronym abstracts (e.g. none of the senses of *CAD*, *CAM*, *CDA*, *DDD* or *DON*).

The accuracy on covered senses looks high (0.85 and 0.83), but it is misleading: many words have only one WordNet sense, so Lesk is "right" without choosing anything. On the words where Lesk really has to choose (*ADP*, *Ca*, *Cilia*, *Nurse*, *Respiration*) it is only slightly better than a random choice (0.617 vs 0.500 on acronyms, 0.534 vs 0.405 on terms). The second selection, made only of words with at least two WordNet senses, separates the two problems more clearly: acronyms still have a low coverage (0.28), so their errors come from WordNet, while terms are almost fully covered (0.95) but Lesk is right on only 45% of the covered abstracts, so their errors come from the algorithm.

TABLE IV
LESK VARIANTS ON THE SECOND SELECTION

Method	Acronyms		Terms	
	Acc.	Acc. cov.	Acc.	Acc. cov.
Standard NLTK Lesk	0.192	0.607	0.403	0.453
MFS (first WordNet sense)	0.233	0.900	0.297	0.347
NLTK Lesk	0.256	0.881	0.478	0.528
Our Lesk: gloss	0.257	0.881	0.493	0.543
Our Lesk: gloss, examples, synonyms	**0.268**	**0.918**	**0.508**	**0.558**

Looking at the errors, we found that some words are disambiguated even worse than by a random choice (*Cement* 0.049, *Milk* 0.040, *RA* 0.000), because Lesk always returns the same wrong sense: *Milk* is "a river in Montana" in 174 of 198 abstracts, *RA* is "right ascension" (astronomy) in 288 of 297 and *US* is "uracil" in 154 of 198. These definitions are long and full of stopwords (*the, in, and, to*), which appear in every abstract and are counted by NLTK as real overlaps.

This suggested some simple improvements, compared in Table IV on the second selection. Removing stopwords is the most useful change (accuracy on covered acronyms from 0.607 to 0.881). Comparing the gloss more carefully (lowercase, no punctuation, no target word) and adding the WordNet examples and synonyms to the signature improve the results further, up to 0.558 on covered terms. Lesk is clearly better than the most frequent sense (MFS) baseline on terms (0.558 vs 0.347), but not on acronyms (0.918 vs 0.900): there, part of the gain comes from the fallback to the first sense when no word overlaps, which helps when the first sense is correct (*US*, *RA*) and hurts when it is not (*Medullary*, from 0.515 to 0.308).

TABLE V
POSITION OF THE CORRECT SENSE IN THE RANKING OF EACH PYWSD METHOD (1 = CORRECT ANSWER; * = TIE FOR FIRST PLACE)

Method	*drug* (3)	*bank* (18)	*ventricle* (2)
Original Lesk	3	1*	1
Simple Lesk	1*	1	1
Adapted Lesk	1*	1	1
Cosine Lesk	1	2	1
Path	1	9	2
Wu–Palmer	1	8	2
Leacock–Chodorow	1	7	2
Resnik	2	10	2*
Jiang–Conrath	1*	2*	2*
Lin	1	8	2*
Max lemma count	1	–	–

C. Task 3: `pywsd` on Example Sentences

We ran the `pywsd` algorithms on three sentences, with target words *drug*, *bank* and *ventricle*. Table V shows, for each method, the position of the correct sense in its ranking, so that we can see not only whether a method is right, but also how far it is from the correct answer.

The Lesk family works best: all its algorithms find the heart ventricle, thanks to the words *heart*, *blood* and *pump*. Original Lesk fails on *drug* because it neither lemmatizes nor removes stopwords: *drugs* does not match *drug*, and a wrong sense wins thanks to the word *to*. Simple and Adapted Lesk are right on *drug*, but only by a tie, since the only shared word is *drug* itself. Cosine Lesk is the only one of the family that misses *bank*, choosing "a container for keeping money" as its first sense.

The similarity-based methods fail on *bank* and *ventricle*. With 18 senses for *bank* the scores are very close and the correct sense is only at position 7–10. For *ventricle* every method prefers the brain ventricle, and Resnik, Jiang–Conrath and Lin even give the same score to both senses. This could happen because these measures only use the *is-a* hierarchy, so they cannot see that the ventricle is *part of* the heart. We also found two problems in the library: Resnik returns the sense with the *lowest* score (the ranking is sorted in ascending order), and Jiang–Conrath gives an infinite score to every sense, because the target word also appears in the context, so it always returns the first sense. Finally, max lemma count ignores the context: since Open English WordNet has no sense frequencies, it always returns the first WordNet sense, which is correct for *drug* but not for *bank* ("sloping land") and *ventricle* (the brain ventricle).

D. Automatic CUI–WordNet Mapping

Before running Task 4 on the full dataset, we validated the automatic mapping against the manual one on the 39 words (Table VI). Precision is the share of automatic links that are also in the manual mapping, and recall is the share of manual links found automatically. Rule 1 alone (a UMLS name equal to a WordNet lemma) is always correct but finds only half of the manual pairs. Adding Rule 2 (a minimum number of shared content words between the UMLS and WordNet descriptions) increases the recall, and this minimum controls the balance between precision and recall: with four content words we obtain the best F1 (0.83), with a precision of 0.89. With this threshold the automatic mapping covers 112 CUIs of 93 words. The remaining errors are few but visible, e.g. the two senses of *Milk* are swapped and one sense of *CAD* is linked to *computer-aided design*.

TABLE VI
VALIDATION OF THE AUTOMATIC MAPPING ON THE 39 MANUALLY MAPPED WORDS

Min. overlap	Precision	Recall	F1	CUIs covered
2	0.73	**0.84**	0.78	135
3	0.82	0.82	0.82	123
4	0.89	0.77	**0.83**	112
Rule 1 only	**1.00**	0.52	0.69	72

E. Tasks 4 and 9: `pywsd` on MSH-WSD

Tables VII and VIII report the results on the 39 selected words, with the manual mapping, and on the full dataset, with the automatic mapping. On the 39 words the best methods reach only 0.257 on acronyms (Path) and 0.371 on terms (Lin). The accuracy on covered senses is much higher on acronyms (about 0.9) than on terms (about 0.55–0.59), but this does not mean that acronyms are easier: only 29% of the acronym abstracts are covered by WordNet, against 74% of the term abstracts, so the high value is computed on a small part of the data, mostly on the few acronyms whose medical meaning is the first WordNet sense.

On the full dataset the coverage is 16% of the acronym abstracts and 40% of the term abstracts, even less than on the 39 words because the automatic mapping finds fewer links than the manual one. For this reason the accuracy is much lower (at most 0.145 on acronyms and 0.250 on terms), since the abstracts whose correct sense has no WordNet synset always count as errors. Again, the high accuracy on covered acronyms (about 0.9) refers to only one acronym abstract in six.

The methods are very close to each other: the differences between the best ones (Simple and Adapted Lesk, Path, Lin, Resnik) are below 0.01 on acronyms and 0.02 on terms, so no method is clearly better. On acronyms, NLTK Lesk, Original Lesk and Max lemma count are the weakest, which confirms that a richer signature and the use of the context help; on terms the ranking is less clear (e.g. NLTK Lesk is above Wu–Palmer on both sets). Jiang–Conrath is also weak on terms (0.284 on the 39 words): as seen in Task 3, its scores are often infinite, so it falls back on the first sense. Resnik as implemented in `pywsd` is the worst method on both sets; after fixing the order of its ranking it becomes one of the best.

F. Task 5: Phonetic Similarity

Fig. 1 shows the two distance matrices of the example sentence. The plain Levenshtein distance mostly reflects the length of the words (*prescribed* and *important* are far from

TABLE VII
PYWSD ALGORITHMS ON THE 39 SELECTED WORDS (MANUAL MAPPING; COVERAGE 0.29 ON ACRONYMS AND 0.74 ON TERMS)

	Acronyms		Terms	
Method	Acc.	Acc. cov.	Acc.	Acc. cov.
NLTK Lesk	0.225	0.750	0.347	0.565
Original Lesk	0.211	0.705	0.334	0.529
Simple Lesk	0.242	0.815	0.353	0.550
Adapted Lesk	0.242	0.815	0.353	0.550
Cosine Lesk	0.230	0.790	0.368	0.568
Path	**0.257**	**0.925**	0.363	0.582
Wu–Palmer	0.250	0.905	0.345	0.562
Leacock–Chodorow	0.249	0.900	0.358	0.576
Resnik (`pywsd`)	0.188	0.680	0.253	0.447
Resnik (fixed)	0.254	0.920	0.339	0.538
Jiang–Conrath	0.242	0.900	0.284	0.465
Lin	0.254	0.920	**0.371**	**0.591**
Max lemma count	0.212	0.750	0.211	0.353

TABLE VIII
PYWSD ALGORITHMS ON THE FULL DATASET (AUTOMATIC MAPPING)

	Acronyms		Terms	
Method	Acc.	Acc. cov.	Acc.	Acc. cov.
NLTK Lesk	0.124	0.773	0.229	0.628
Original Lesk	0.126	0.795	0.205	0.554
Simple Lesk	0.143	0.905	0.249	0.656
Adapted Lesk	0.143	0.905	**0.250**	0.658
Cosine Lesk	0.133	0.848	0.223	0.609
Path	**0.145**	**0.932**	0.244	**0.670**
Wu–Palmer	0.141	0.911	0.222	0.619
Leacock–Chodorow	0.140	0.897	0.236	0.652
Resnik (`pywsd`)	0.107	0.682	0.161	0.458
Resnik (fixed)	0.143	0.924	0.237	0.644
Jiang–Conrath	0.140	0.909	0.211	0.573
Lin	0.143	0.924	0.240	0.662
Max lemma count	0.135	0.864	0.196	0.533

every other word), while the Soundex distance is bounded by the length of the code (0–4) and groups words that sound alike. For example *two* and *to* have the same code (distance 0), which is phonetically correct but shows that Soundex cannot distinguish homophones with different meanings. Many other pairs (35 of 78) have the maximum distance, because their first letters, which Soundex keeps unchanged, are different. We also computed the matrices for translations of the sentence into six other languages (Fig. 2): Soundex was designed for English names, so its codes are less meaningful for the other languages, however, when germanic words were analyzed, we could clearly see some analogies to the english version. Also, all latin-derived languages (such as italian and spanish), are similar to each others.

G. Tasks 6 and 9: Phonetic–Semantic Hybrid

Before the evaluation, we tested the hybrid similarity on the sentence of Task 1. With $\alpha = 1$ (only Wu–Palmer) the correct sense of *drug* wins, while with $\alpha = 0.5$ the verb "administer a drug to" is chosen: one of its lemmas, *dose* (D200), has a short Soundex code close to those of short context words such as *two*, *to* and *today* (T000, T300), so it receives a small phonetic bonus from many words, only because of their sound.

TABLE IX
WU–PALMER AND HYBRID SIMILARITY ON THE 39 WORDS

	Acronyms		Terms	
Method	Acc.	Acc. cov.	Acc.	Acc. cov.
Wu–Palmer (`pywsd`)	0.250	0.905	**0.345**	**0.562**
Hybrid, $\alpha = 1$	0.248	0.895	**0.345**	**0.562**
Hybrid, $\alpha = 0.8$	0.249	0.895	0.332	0.547
Hybrid, $\alpha = 0.5$	**0.265**	**0.920**	0.342	0.559

TABLE X
WU–PALMER AND HYBRID SIMILARITY ON THE FULL DATASET

	Acronyms		Terms	
Method	Acc.	Acc. cov.	Acc.	Acc. cov.
Wu–Palmer (`pywsd`)	**0.141**	**0.911**	0.222	0.619
Hybrid, $\alpha = 1$	0.130	0.835	0.221	0.618
Hybrid, $\alpha = 0.8$	0.131	0.835	**0.232**	**0.637**
Hybrid, $\alpha = 0.5$	0.134	0.842	0.231	0.631

Table IX compares the Wu–Palmer method of `pywsd` with our hybrid similarity for three values of α on the 39 words. With $\alpha = 1$ our implementation gives almost the same results as `pywsd`, which validates it. With the required $\alpha = 0.5$ the accuracy on acronyms increases slightly (from 0.248 with $\alpha = 1$ to 0.265), while on terms it stays almost the same (0.345 to 0.342). The phonetic component changes the result on only 13 of the 39 words, and it helps on some and hurts on others. It helps on two acronyms, *ADP* (0.70 to 0.85) and *PVC* (0.60 to 0.85): since a Soundex code has only four characters, the code of a long name usually depends only on its first word (*adenosine diphosphate* has the code of *adenosine*), so when the abstract contains that word the correct sense gets a phonetic similarity of 1. It hurts on terms such as *ERUPTION* (0.25 to 0.00) and *Respiration* (0.80 to 0.65), whose senses have the same name: there the phonetic part cannot separate them and only adds noise. In short, the phonetic similarity helps only when the name of the correct sense contains a word of the context, which is a form of word overlap rather than of phonetic similarity.

On the full dataset (Table X) the picture is similar. The phonetic component improves terms by about one point with respect to $\alpha = 1$ (0.221 to 0.231) and acronyms only slightly (0.130 to 0.134). Our Wu–Palmer implementation ($\alpha = 1$) is a little below the one of `pywsd` on acronyms (0.130 vs 0.141), probably because our implementation removes the target word from the context, while `pywsd` keeps it; for this reason the fair comparison for the phonetic component is with $\alpha = 1$.

H. Task 10: Phonetic Lesk

We first tested the three word-matching rules on pairs of words. Levenshtein (distance $\leq$ 1, words of at least five

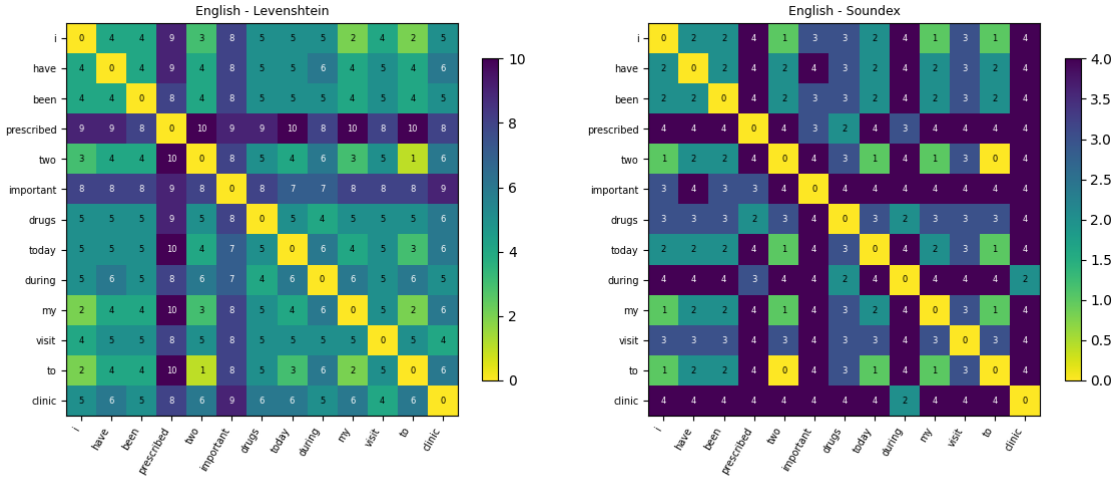


Fig. 1. Levenshtein distance between the words (left) and between their Soundex codes (right) for the sentence of Task 1.

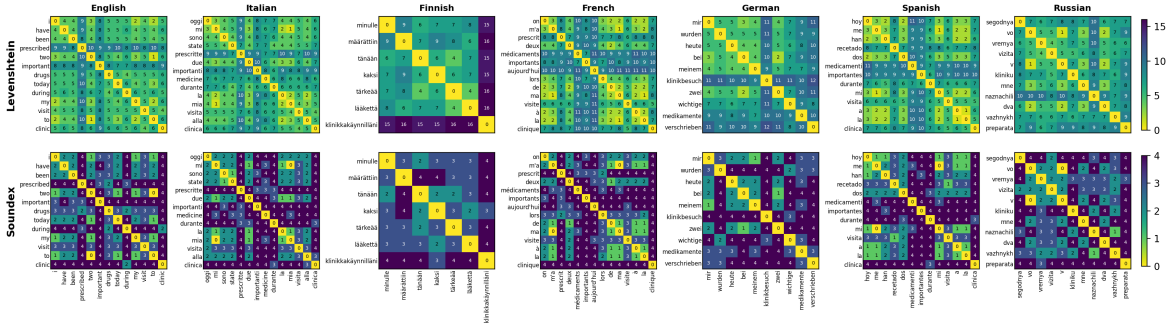


Fig. 2. Same analysis carried out in Fig.1, on the same sentence translated in 7 different languages

TABLE XI
LESK WITH THREE WORD-MATCHING RULES ON THE 39 WORDS

	Acronyms		Terms	
Matching	Acc.	Acc. cov.	Acc.	Acc. cov.
Exact	**0.255**	**0.875**	0.429	0.656
Soundex	0.242	0.820	0.403	0.621
Levenshtein	0.252	0.865	**0.439**	**0.668**

letters) handles all of them correctly. Soundex instead joins unrelated words (*drug*/*dirge*, *heart*/*hurt*, *cell*/*cello*) and misses real variants whose first letter changes (*oedema*/*edema*).

Table XI shows the results on the 39 words (810 abstracts). Soundex makes Lesk worse on both groups, because its wrong matches give points to wrong senses. Levenshtein slightly improves terms (0.429 to 0.439) and is almost the same on acronyms. The effect is small: Levenshtein changes the answer in only 38 of the 810 abstracts (10 better, 8 worse, 20 wrong in both cases). Interestingly, *Respiration* and *ERUPTION*, which got worse with the Soundex similarity in Task 6, improve here.

I. Task 11: Replacing Wordnet with SapBERT

Since SapBERT compares the context directly with UMLS concepts, it needs no WordNet mapping and can be evaluated on all 200 words. As shown in Table XII, it reaches 0.875 on acronyms and 0.711 on terms, far above every WordNet-based method and above a random choice (about 0.49). The ranking of the two groups is reversed with respect to Lesk: the senses of an acronym are usually very different (*CAD*: coronary artery disease or computer-aided diagnosis) and easy to separate with the right descriptions, while the senses of a term are often close (*Cilia*: cell cilia or eyelashes; *Milk*: milk or human milk).

The variants show that a larger context does not help (the ten closest words already contain the useful information), and that combining names and definition is better than using only one of them (about 4 to 6 points more).

J. Task 7: Graphical Interface

To develop the GUI, we decided to use the django package, together with our python scripts and notebooks. Given that the main goal was to develop a working, responsive website-like interface, compatible also with mobile devices, the choice for the frontend was the Bootstrap5 CDN, limiting as much as we

TABLE XII
SAPBERT RESULTS (ACCURACY)

Setting	Acronyms	Terms
All 200 words, window ±10, names + def.	**0.875**	0.711
39 words, window ±10, names + def.	0.844	0.718
All words, window ±25	0.841	**0.719**
All words, window ±50	0.839	0.715
All words, window ±25, names only	0.806	0.678
All words, window ±25, definition only	0.788	0.660

could Javascript usage. Along with the traditional input bar, the interface allows also to choose which algorithm can be implemented. To speed up the algorithms, when the server is turned on, the models get immediately loaded into the cache, to reduce response times for every query.

K. Limitations

The main limitation of the WordNet-based methods is the mapping between UMLS and WordNet: it was built by hand for 39 words, some choices are subjective (e.g. *Follicles*, *Milk*, *Respiration*), and the automatic mapping used for the full dataset makes some mistakes (e.g. the two senses of *Milk* are swapped). The selections of Task 2 contain only 20 words each, so the averages are sensitive to single words. Finally, SapBERT is based on a model pre-trained on PubMed abstracts, so it may have already seen the texts of MSH-WSD, although without their sense labels.

IV. OVERALL DISCUSSIONS

The main lesson of the project is that the knowledge source matters more than the algorithm. Across Tasks 2, 4 and 6, all the WordNet-based methods stay in the same range (about 0.25 on acronyms and 0.35–0.51 on terms on the selected words, and even less on the full dataset, where the automatic mapping covers fewer senses), whatever the scoring function: on the full dataset the best `pywsd` methods differ by less than 0.02. The difference between acronyms and terms is also explained by WordNet: most medical acronyms are simply not in WordNet, so even a perfect algorithm could not disambiguate them, while for terms the senses are available but the algorithms often choose wrongly. The literature confirms this view: on the same dataset, a Lesk-like method that uses UMLS definitions instead of WordNet glosses reaches about 0.81 accuracy [11], and comparing context embeddings with UMLS definitions reaches about 0.84 [13]. Our SapBERT experiment follows the same idea and obtains 0.875 on acronyms and 0.711 on terms (about 0.80 on average over the 200 words, weighting the two groups by their size), in the same range as these works, although the comparison is not exact, since we use a window of ±10 words and average the accuracy per word. All these knowledge-based results remain below the supervised classifiers reported on MSH-WSD (about 0.94 [11]), which learn from labelled abstracts of each word: this is the price of a method that, like ours, needs no training data for the word to disambiguate.

Within the WordNet-based methods, however, simple fixes matter. The error analysis of Task 2 showed that many errors of NLTK Lesk are systematic and come from implementation details rather than from the idea of Lesk: stopwords counted as overlaps, punctuation attached to the words of the gloss, and the fallback to the first WordNet sense when no word overlaps. The same problem appears already in Task 1, where NLTK Lesk chooses "administer a drug to" for *drugs* only because of the stopword *to*. Removing stopwords and enriching the signature with examples and synonyms, as done by the extended versions of Lesk [6], gave the largest improvements of the project among WordNet-based methods. Looking at the code of the libraries was also necessary: both `nltk.wsd.lesk` and the *Original Lesk* of `pywsd` actually implement the simplified Lesk [5], and two similarity methods of `pywsd` (Resnik and Jiang–Conrath) do not behave as expected.

Finally, our main original contribution, the use of phonetic similarity, did not improve WSD. We tested it both inside the Wu–Palmer similarity (Task 6) and as an approximate word matching inside Lesk, and in both cases the effect is small and not consistent. Soundex codes are too simple: they keep only a few letters, so unrelated words receive the same code, as the distance matrix of Task 5 already shows (*two* and *to* share a code, and Soundex, designed for English names, is even less meaningful for the translations of the sentence). When the phonetic part helps, it is because the name of the correct sense contains a word of the context, which is a form of word overlap rather than of phonetic similarity. A careful edit distance helps only a little, because spelling variants (e.g. British and American spelling) are not frequent in edited abstracts. Although negative, this result is informative: phonetic matching is designed for names and noisy text, and the meaning of a word is not related to its sound.

V. CONCLUSION AND PERSPECTIVES

In this project we studied knowledge-based Word Sense Disambiguation on the MSH-WSD biomedical dataset. Already on a single example sentence, the standard Lesk of NLTK chooses the wrong sense of *drug* because of a stopword, showing how fragile word overlap is on short contexts. On MSH-WSD, classical algorithms based on WordNet, from the standard Lesk of NLTK to the methods of `pywsd`, reach at most about 0.27 accuracy on acronyms and 0.51 on terms on the selected words, and only 0.15 and 0.25 on the full dataset, mainly because WordNet does not contain most medical meanings; the differences between the algorithms are much smaller than the loss caused by the missing senses. Cleaning the input and enriching the sense signatures improve Lesk, while adding phonetic similarity, both to Wu–Palmer and inside Lesk, gives only small and inconsistent changes, as the phonetic distance matrix of the example sentence already suggested: Soundex groups words by sound, not by meaning. A neural approach based on SapBERT and UMLS descriptions, which does not need WordNet nor any training on the dataset, reaches 0.88 on acronyms and 0.71 on terms over all 200 words.

The most difficult part of the work was not running the algorithms, but making their answers comparable with the gold labels: the correct senses are UMLS concepts, while the algorithms answer with WordNet synsets, so we had to build and validate a mapping between the two, first by hand and then automatically. Reading the source code of the libraries was also necessary to understand some unexpected results. Through the project we gained practical experience with WordNet, UMLS and the UMLS REST API, the NLTK and `pywsd` libraries, phonetic algorithms, transformer embeddings, web development with Django and Bootstrap, and the evaluation of NLP systems, in particular the importance of baselines and error analysis.

As a next step, the most promising direction is to rely on biomedical knowledge instead of WordNet. A first, simple experiment would be to run our Lesk with the UMLS definitions as sense signatures: comparing it with the WordNet version would separate the effect of the algorithm from the effect of the knowledge source. SapBERT could also be improved by using a few labelled examples per sense, and a large language model could be asked to choose among the UMLS descriptions. Finally, phonetic similarity could be more useful where words are really misspelled, for example in clinical notes or in transcribed speech, rather than in edited scientific abstracts.

Use of AI

We used an AI assistant (Claude, by Anthropic) as a support tool during the project. It was used to improve grammar and technical writing, but all experiments and analyses were carried out by the authors. It was used to explain concepts and library behaviour (e.g. the algorithms of NLTK and `pywsd`), to review and correct code that we wrote step by step, to fix glitches (when they occurred), to suggest the structure of some experiments, to search for relevant literature, and to draft and revise parts of the text of this report and of the presentation. Some graphical choices were helped in the development by Claude Opus 5 (helped in the use of some flex-based tools of Bootstrap5, to speed up the process of adapting the web-app to a mobile working-version). All the experiments were run by us, all the results reported come from our own code and data, and we checked every reference and every number in the report. The design choices, the analysis of the results and the final text are our responsibility.

References

[1] R. Navigli, "Word sense disambiguation: A survey," *ACM Computing Surveys*, vol. 41, no. 2, 2009.

[2] M. J. Schuemie, J. A. Kors, and B. Mons, "Word sense disambiguation in the biomedical domain: An overview," *Journal of Computational Biology*, vol. 12, no. 5, pp. 554–565, 2005.

[3] O. Bodenreider, "The Unified Medical Language System (UMLS): Integrating biomedical terminology," *Nucleic Acids Research*, vol. 32, no. Database issue, pp. D267–D270, 2004.

[4] M. Lesk, "Automatic sense disambiguation using machine readable dictionaries: How to tell a pine cone from an ice cream cone," in *Proc. 5th Annual International Conference on Systems Documentation (SIGDOC)*, 1986, pp. 24–26.

[5] A. Kilgarriff and J. Rosenzweig, "Framework and results for English SENSEVAL," *Computers and the Humanities*, vol. 34, 2000.

[6] S. Banerjee and T. Pedersen, "An adapted Lesk algorithm for word sense disambiguation using WordNet," in *Proc. CICLing*, 2002, pp. 136–145.

[7] Z. Wu and M. Palmer, "Verb semantics and lexical selection," in *Proc. 32nd Annual Meeting of the ACL*, 1994, pp. 133–138.

[8] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly, 2009.

[9] "pywsd: Python implementations of word sense disambiguation technologies," https://github.com/alvations/pywsd, accessed: 2026-10-20.

[10] G. A. Miller, "WordNet: A lexical database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[11] A. J. Jimeno-Yepes, B. T. McInnes, and A. R. Aronson, "Exploiting MeSH indexing in MEDLINE to generate a data set for word sense disambiguation," *BMC Bioinformatics*, vol. 12, p. 223, 2011.

[12] A. J. Jimeno-Yepes and A. R. Aronson, "Knowledge-based biomedical word sense disambiguation: Comparison of approaches," *BMC Bioinformatics*, vol. 11, p. 569, 2010.

[13] S. Tulkens, S. Šuster, and W. Daelemans, "Using distributed representations to disambiguate biomedical and clinical concepts," in *Proc. 15th Workshop on Biomedical Natural Language Processing (BioNLP)*, 2016.

[14] F. Liu, E. Shareghi, Z. Meng, M. Basaldella, and N. Collier, "Self-alignment pretraining for biomedical entity representations," in *Proc. NAACL-HLT*, 2021.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[16] R. C. Russell, "Soundex," U.S. Patent 1,261,167, 1918.

[17] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[18] C. Leacock and M. Chodorow, "Combining local context and WordNet similarity for word sense identification," in *WordNet: An Electronic Lexical Database*. MIT Press, 1998.

[19] P. Resnik, "Using information content to evaluate semantic similarity in a taxonomy," in *Proc. IJCAI*, 1995.

[20] J. J. Jiang and D. W. Conrath, "Semantic similarity based on corpus statistics and lexical taxonomy," in *Proc. ROCLING X*, 1997.

[21] D. Lin, "An information-theoretic definition of similarity," in *Proc. ICML*, 1998.

[22] J. P. McCrae, A. Rademaker, F. Bond, E. Rudnicka, and C. Fellbaum, "English WordNet 2019 – an open-source WordNet for English," in *Proc. 10th Global WordNet Conference*, 2019.